HW1 — Logic, SAT/SMT, and bounded reachability

CS 3892 / CS 5892 · Formal Methods: Safe and Trustworthy AI · Fall 2026

Out	Thursday, September 10
Due	**Thursday, September 24, 11:59 p.m. US Central**
Weight	4% of the course grade (100 points)
Tool	Z3 — Python API and SMT-LIB 2
Zero-install path	Google Colab, or GitHub Codespaces
Submit	Brightspace → Assignments → *HW1 — Logic; SAT-SMT; bounded reachability (Z3)*

Nothing to install. Every part of this runs in a browser tab.

Start here: github.com/ttj/cs3892-examples has a starter notebook for this homework with each part stubbed out, plus every worked example from sessions 5 onward in both SMT-LIB and Python.

What this is for

By the end you should be able to take an English requirement, write it down as a formula, decide it with a solver, and — the part that matters — **know whether the answer you got is the answer to the question you meant to ask.** A wrong encoding that returns `unsat` looks exactly like a correct one.

Part 1 — Logic warm-up *(20 points)*

Translate five English statements into propositional and, where they need it, first-order logic. For each one, **decide satisfiability and validity by hand first**, then confirm with Z3.

Doing it both ways is the point: you are calibrating your own judgement against a decision procedure. Where you and Z3 disagreed, say so and say who was right.

Recall the identity everything here rests on:

> φ is **valid** exactly when ¬φ is **unsatisfiable**.

A solver has no "is it valid?" button, so every validity question you ask becomes a satisfiability question about the negation.

Part 2 — Encoding *(30 points)*

Encode a combinatorial puzzle as an SMT query: pigeonhole, n-queens, sudoku, or KenKen. Starters are provided for each.

Then **report solve times as the size grows, and say where the wall is.** A table and one sentence of interpretation. "It got slow around n = 12" is not an answer; "it goes from milliseconds to minutes between n = 11 and n = 12, which is where the search stops fitting in the clause database" is closer.

Part 3 — Bounded reachability *(30 points)*

Write the bounded counter as a transition system and ask Z3 whether `x` can reach 11 within `N` steps.

Find the smallest `N` for which the answer changes, and explain precisely why that `N`. This is the running example for the whole semester — you will verify this same property four more times with four more tools, so it is worth understanding completely now.

Part 4 — AI component *(20 points)*

Ask an LLM to produce the SMT-LIB encoding for Part 3. Report what it got right, what it got wrong, and — critically — **how you knew.**

An encoding that is `unsat` for the wrong reason looks exactly like a correct one until you check it. Say what check you ran. "I read it and it looked fine" is not a check; asserting the negation of a property you know should hold, and watching it come back `sat` with a model you can read, is.

What to submit

One archive or repository link containing:

1. Your `.py` and/or `.smt2` files, runnable as-is.
2. A short report — Markdown or PDF, roughly two pages — with the answers, the timing table, and your Part 4 write-up.
3. The AI-use disclosure (see below).

The mechanical part is checked automatically: do your files run, and do they produce the verdicts you claim? The interesting part — why the counterexample looks like that — is read by a human.

Late policy

Four late days for the semester, in whole-day units, no more than two on any one homework, no questions asked. Beyond that, 20% per day.

Collaboration and AI use

Discussion is encouraged; submitted work must be your own. **AI assistants are allowed and expected** — this is a course about the reliability of AI-produced artifacts. Two conditions:

1. **Disclose.** Say what you used and for what.
2. **Verify.** You are responsible for everything you submit. "The model said so" is not a justification, and here it is a particularly poor one: every tool in this course is a decision procedure that can tell you whether the model was right. Use it.